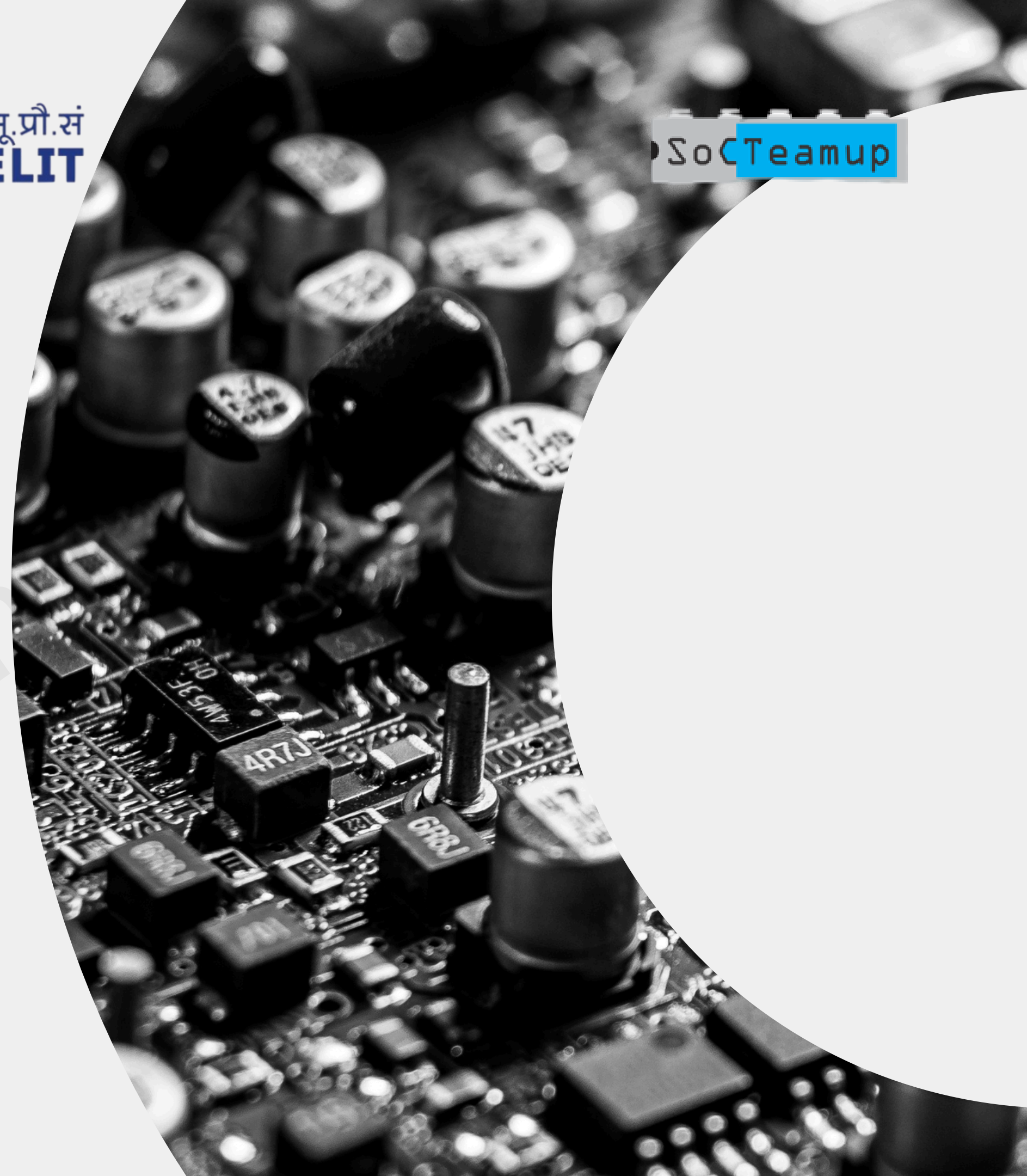


Introduction to RTL to GDSII Flow

echiph





Outline

Introduction to the RTL → GDSII Flow
Overview of Front-End vs Back-End
Step 1: RTL Synthesis (Yosys)
Step 2: Floorplanning (io_placer in Openlane)
Step 3: Placement (RePLACE in Openlane)
Step 4: Clock Tree Synthesis (TritonCTS in OpenLane)
Step 5: Routing (RePLACE , OpenDP in OenLane)
Step 6: Layout Generation (Magic)
Step 7: Netlist Verification (Netgen)
Example Flow Summary
Key File Formats
Summary and Best Practices

➔ What is the RTL to GDSII Flow?

- RTL to GDSII describes the complete digital VLSI design flow that converts a Verilog design into a chip layout ready for fabrication.
- It involves multiple front-end and back-end processes.

➔ What is the RTL to GDSII Flow?

Stage	Domain	Description
RTL Design	Front-End	Functional Verilog code
Synthesis	Front-End	Convert RTL → gates
Floorplan to GDSII	Back-End	Convert gates → layout polygons

Goal: Generate a verified, manufacturable layout that meets area, timing, and power constraints.

➔ Open-Source Toolchain Overview

Design Stage	Tool Used	Output File	Description
RTL Synthesis	Yosys	Gate -level Netlist (.v)	Converts RTL to logic gates
Floorplanning	OpenLane(init_fp, ioPlacer)	Floorplan DEF	Defines die/core, pins, power grid
Placement	OpenLane (RePlAcE engine)	Placed DEF	Positions standard cells
Clock Tree Synthesis	OpenLane (TritonCTS)	Updated DEF	Adds buffers for balanced clock
Routing	OpenLane (FastRoute, TritonRoute)	Routed DEF	Connects all placed cells
Layout Generation	Klayout	GDSII	Generates fabrication layout
DRC Checks	Magic	DRC Report	Checks design rule Violations
LVS	Netgen	Report	Compares layout vs schematic
Timing Analysis	OpenSTA	Timing report	Perform static timing analysis

➔ Step 1: RTL Synthesis (Yosys)

Purpose: Convert Verilog RTL into a gate-level netlist.

Tool: Yosys (open-source synthesis engine)

Example:

```
read_verilog mux2.v
synth -top mux2
abc -g simple
write_verilog
mux2_netlist.v
stat
```

Outputs:

- mux2_netlist.v → synthesized structural netlist
- stat report → cell and wire summary

Step 1: RTL Synthesis (Yosys)

Terms:

- **Synthesis:** logical transformation from RTL → gates
- **Netlist:** structural Verilog describing logic interconnections

➔ Example: RTL to Gate-Level

- RTL (mux2.v):

```
assign y = sel ? d1 : d0;
```

- After Synthesis (Yosys output):

```
assign y = (sel & d1) | (~sel & d0);
```

- Behavioral “?” reduced to logic gates (AND, OR, NOT).
- This gate-level form is what will be physically implemented.

➔ Step 2: Floorplanning

- Purpose: Define chip size, core area, pin locations, and power grid before placement.
- Tool: OpenLane (init_fp and ioPlacer)

```
run_floorplan
```

Outputs:

- `Design_name_floorplan.def` → graphical layout view
- Defines boundaries for placement and routing.

Step 3: Placement (RePLace)

- **Purpose:** Place synthesized standard cells optimally within the defined core area.
- **Tool:** RePLAce(cell placement)
- **Input:**
 - Synthesized netlist (.v)
 - Standard-cell LEF (.lef)
 - Floorplan constraints (Config.json)

➔ Step 3: Placement (RePLace)

- **Example config:**

Design_name_config.json

Outputs:

- Placement.def → cell coordinates and placement rows

Term:

- **Legal placement:** cells aligned to valid sites without overlap.

Step 4: Clock Tree Synthesis (CTS)

- **Purpose:** Add clock buffers/inverters to ensure the clock reaches all flip-flops simultaneously.
- **Tool:**
 - Custom Tcl or Python CTS script (in open-source flows).
Example flow.tcl
 - Uses cell insertion based on fanout and timing.

Step 4: Clock Tree Synthesis (CTS)

- **Simple Script Example:**

```
foreach cell [get_cells -clock_pins] {  
  insert_buffer -cell CLKBUF -net clk  
}
```

Outputs:

- Updated netlist / DEF with inserted buffers
- Cts.def
- Report of maximum skew and clock latency

New Terms:

- **Clock skew:** timing difference between clock arrival at two flip-flops
- **Insertion delay:** total delay from clock source to sink

➔ Step 5: Routing (FastRoute, TritonRoute)

- **Purpose:** Connect all pins and cells using metal layers and vias according to design rules.
- **Tool:** FastRoute TritonRoute
- **Input:**
 - Placed DEF
 - Technology LEF
 - Netlist

➔ Step 5: Routing (FastRoute, TritonRoute)

- **Example:**

```
run_routing
```

Outputs:

- Design_name_routed.def → routing database
- Wire connection list adhering to DRC

New Term:

- Routing layer: metal layer used for interconnect
- Via: connection between two metal layers

➔ Step 6: Layout Generation (Magic)

- **Purpose:** Generate the final mask-level layout and export GDSII for fabrication.
- **Tool:** Magic

Command Sequence:
magic576 design_name.mag

Outputs:

- Design_name.gds → complete geometric layout
- Design_name.mag → Magic internal format

New Term:

- **GDSII:** standard IC layout format sent to fabrication.

➔ Step 7: Layout vs Schematic Check (Netgen)

- **Purpose:** Ensure that the layout connectivity matches the synthesized netlist.
- **Tool:** Netgen

➔ Step 7: Layout vs Schematic Check (Netgen)

- **Example Command:**

```
netgen -batch lvs "design.spice design"  
"design_netlist.spice design" setup.tcl
```

Outputs:

- LVS report with matched / unmatched nodes

Terms:

- **LVS (Layout vs Schematic):** verifies logical equivalence between schematic (netlist) and layout.
- **DRC (Design Rule Check):** ensures minimum spacing, width, and overlap rules are met.

➔ Where Does **Resistance** Come From?

Stage	Tool	Command	Output
Synthesis	Yosys	yosys -s run.ys	design_netlist.v
Floorplan	Init_fp , io_Placer	Run_floorplan	Floorplan.def
Placement	RePlace	Run_placement	placed.def
CTS	TritonCTS	Run_cts	cts.def
Routing	FastRoute, TritonRoute	Run_routing	routed.def
DRC/LVS	Magic / Netgen	Run_magic_drc/ Run_lvs	Report
GDS Export	Magic/klayout	gds write design.gds	Final GDSII

➔ Common File Formats

File Type	Description	Tool(s)
.v	RTL/Netlist	Yosys
.lef	Library Exchange Format	klayout
.def	Design Exchange Format	klayout
.mag	Magic layout format	Magic
.gds	Final layout	Magic,klayout
.spice	Extracted circuit	Magic / Netgen

Integration with Open PDKs

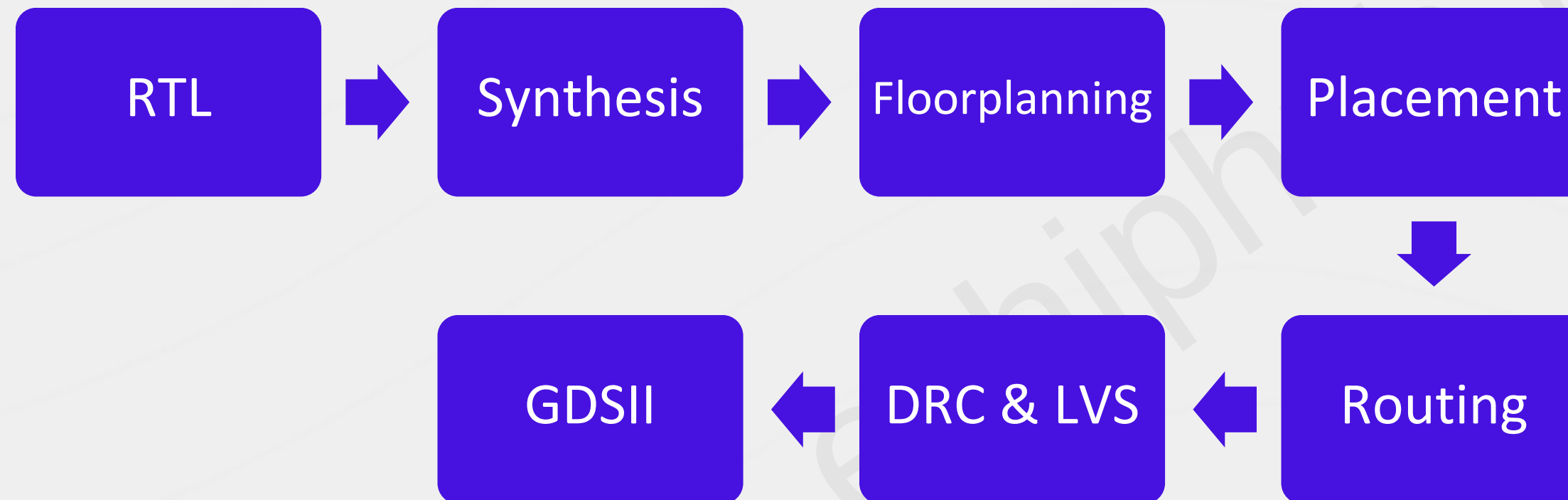
PDK (Process Design Kit) contains:

- Layer information (metal, via, diffusion)
- Design rules (DRC)
- Standard-cell LEF/Liberty files
- Technology file for Magic

Example:

- SkyWater 130nm PDK (sky130A) – fully open and compatible with:
 - Yosys synthesis
 - Openlane
 - Magic_layout

➔ Example Directory Structure



Key Learnings

- **Yosys → Openlane → Magic → Netgen** together form a complete open-source RTL-to-GDSII flow.
- Each stage refines the design with more physical details.
- Standard formats (LEF, DEF, GDSII) ensure interoperability.
- Verification steps (DRC, LVS) ensure layout correctness before fabrication.

➔ Summary and Best Practices

Summary

RTL → Netlist → Floorplan → Placement → CTS → Routing → GDSII

The GDSII output is fabrication-ready once DRC/LVS pass.

Each stage uses distinct open-source tools for clarity and learning

Best Practices

Keep design utilization $\leq 70\%$ to avoid routing congestion.

Run DRC and LVS checks after routing.

Cross-check parasitic data (.spef) before final signoff.

Start with small cells (like ALUs, muxes) before SoC-level blocks

Thank You

echiphu

